

Roteiro do Trabalho, Dia 1: banco de dados e API funcionando

Trabalho individual ou em dupla. Você já resolveu as queries desse projeto no bdc2 (tarefas ecom01 a ecom19). Aqui o objetivo é outro: pegar aquelas queries prontas e transformar num projeto de verdade rodando na sua máquina, do banco até a tela. Dois dias: no primeiro a API tem que responder tudo certo, no segundo o site tem que consumir a API sem erro nenhum. Este documento cobre o Dia 1.

Se travar em alguma etapa, teste sempre no menor pedaço possível antes de seguir. Testar uma query no DBeaver antes de colar no Flask. Testar uma rota no navegador ou no Insomnia antes de chamar ela do JavaScript. Isso evita ficar procurando erro no lugar errado.

Conceitos que você vai usar hoje

- **Banco de dados relacional:** um jeito de guardar dados em tabelas que se relacionam entre si. Um produto pertence a uma categoria, um pedido pertence a um cliente, e assim por diante.
- **Tabela, coluna, linha:** uma tabela é como uma planilha com nome fixo de colunas (a tabela produtos tem as colunas nome, preco, estoque). Cada linha é um registro, um produto específico.
- **Chave primária (PK):** a coluna que identifica cada linha de forma única dentro da tabela. Quase sempre chamada de id.
- **Chave estrangeira (FK):** uma coluna que guarda o id de uma linha de outra tabela, criando a ligação entre elas. produtos.categoria_id aponta pra categorias.id.
- **DDL (definição):** os comandos SQL que criam a estrutura do banco. O principal é CREATE TABLE.
- **DML (manipulação):** os comandos SQL que leem ou alteram dados. SELECT lê, INSERT cria, UPDATE altera, DELETE remove.
- **Terminal:** a janela de linha de comando (Terminal no Mac/Linux, PowerShell no Windows). Você digita um comando, aperta Enter, ele roda e mostra o resultado embaixo.
- **Git e GitHub:** git guarda o histórico de versões do seu código na sua máquina. GitHub é o site que hospeda esse histórico na nuvem. Commit é salvar um ponto nesse histórico. Push é enviar esses commits pro GitHub. Fork e clone estão explicados no Passo 0, abaixo.
- **API:** um programa que fica esperando pedidos (requisições) e devolve respostas em JSON, sem tela nenhuma. O Flask que você vai completar hoje é a API da loja.
- **Rota (endpoint):** um endereço específico da API, por exemplo /api/produtos. Cada rota faz uma coisa.
- **HTTP GET e POST:** GET pede dados (listar produtos). POST manda dados pra criar algo (cadastrar um cliente).

- **JSON:** o formato de texto que a API usa pra devolver dados. Parecido com um dicionário Python: {"nome": "Mouse", "preco": 49.9}.
- **Código de status HTTP:** um número que a resposta traz dizendo o que aconteceu. 200 é sucesso, 201 é "criado com sucesso", 400 é erro no que foi pedido, 404 é "não encontrado", 500 é erro no servidor.
- **Ambiente virtual (venv):** uma pasta isolada com a própria cópia do Python e das bibliotecas do projeto, pra não misturar com o resto do computador. Você ativa ele antes de instalar ou rodar qualquer coisa daqui.

Passo 0: pegar o projeto no GitHub

1. Abra o repositório do professor no navegador:
`https://github.com/<USUARIO-DO-PROFESSOR>/trab-bd`
2. Clique em **Fork** (canto superior direito) e crie o fork na sua conta. Não marque a opção de copiar só a branch main, quer o repositório completo.
3. Clone o SEU fork, não o do professor:

```
git clone https://github.com/<SEU-USUARIO>/trab-bd.git
cd trab-bd
```

4. Confirme que o remote é o seu fork:

```
git remote -v
```

5. Se o professor atualizar o repositório original depois, você busca as atualizações assim (só rode isso se for pedido):

```
git remote add upstream https://github.com/<USUARIO-DO-PROFESSOR>/trab-bd.git
git fetch upstream
git merge upstream/main
```

Comite ao longo dos dois dias, não só no final. Cada bloco deste roteiro termina com um commit sugerido. Sem commit intermediário não tem como provar que o trabalho foi feito por etapas.

1. Escolher o tema

Nome da loja, categorias, e o tipo de produto que ela vende. A estrutura de tabelas é igual pra todo mundo (contrato em `docs/especificacao-banco.md`), só o conteúdo muda.

2. Montar o schema.sql

Pasta: `material-alunos/database/`. Leia `LEIA-ME.md` dessa pasta primeiro.

Nas tarefas `ecom01`, `ecom02` e `ecom03` do `bdc2` você já escreveu os `CREATE TABLE` de `categorias/produtos`, `clientes/cupons` e `pedidos/itens_pedido/avaliacoes`. Pegue essas soluções e cole todas juntas num único arquivo `schema.sql`. Os nomes de tabela e coluna têm que ficar

exatamente iguais ao contrato, isso não muda com o tema. O que você pode ajustar é adicionar colunas extras opcionais do seu tema (aceitando `NULL` ou com `DEFAULT`), se quiser.

Não esqueça `PRAGMA foreign_keys = ON;` no topo do arquivo.

3. Montar o seed.sql

Mesma pasta. Popule com o tema escolhido: pelo menos 5 categorias, 20 produtos, 5 clientes, 3 cupons (um vencido ou inativo), 5 pedidos históricos com itens, e 10 avaliações. Sem pedido histórico os relatórios do painel ficam vazios e você não vai conseguir testar nada no fim do dia.

4. Criar o banco com o DBeaver

O DBeaver é um programa com tela (não é terminal) pra criar bancos, escrever SQL e ver o resultado numa tabela na hora. É mais fácil de enxergar o que está acontecendo do que digitando comando por comando.

Instalar: baixe o DBeaver Community (gratuito) em <https://dbeaver.io/download> e instale. Abra o programa.

Criar a conexão com o banco da loja:

1. Clique no ícone de tomada no canto superior esquerdo (ou menu **Database > New Database Connection**).
2. Escolha **SQLite** na lista e clique em **Next**. Se for a primeira vez usando SQLite no DBeaver, ele vai avisar que falta um driver, clique em **Download** e espere terminar.
3. No campo do caminho do arquivo, navegue até a pasta do projeto que você clonou e aponte para `material-alunos/database/loja.db`. O arquivo ainda não existe, o DBeaver cria ele quando você terminar.
4. Clique em **Finish**. A conexão aparece na árvore à esquerda, ainda sem tabela nenhuma.

Rodar o schema.sql:

5. Botão direito na conexão > **SQL Editor > New SQL Script** (ou o ícone de folha com um `+`). Abre uma aba em branco.
6. Cole ali todos os `CREATE TABLE` que você escreveu no passo 2, com o `PRAGMA foreign_keys = ON;` no topo.
7. Execute o script inteiro (não linha por linha): `Alt+X` no Windows/ Linux, `Option+X` no Mac, ou o ícone de "Execute SQL Script" com várias setas. Erro de sintaxe aparece embaixo, indicando a linha.
8. Atualize a árvore (botão direito na conexão > **Refresh**, ou tecla F5) e abra **Tables**. As tabelas que você criou têm que aparecer ali.

Rodar o seed.sql:

9. Abra outra aba de SQL Editor (repita o passo 5), cole todos os `INSERT` que você escreveu no passo 3, e execute o script inteiro de novo.

10. Clique duas vezes em qualquer tabela (por exemplo `produtos`) pra ver os dados numa grade, sem precisar escrever `SELECT`.

Agora sim, salve os arquivos de verdade. O DBeaver só serviu pra escrever e testar visualmente. O que importa pro projeto (e pro commit) é o texto salvo em arquivo. Copie exatamente o SQL que você rodou em cada aba e salve em:

- `material-alunos/database/schema.sql`, OS `CREATE TABLE`
- `material-alunos/database/seed.sql`, OS `INSERT`

Confirme que os arquivos batem com o banco que você testou. No terminal:

```
cd material-alunos/database
python3 init_db.py
```

Esse script apaga o `loja.db` e recria ele do zero, só a partir dos dois arquivos `.sql`. Se der erro aqui e não tinha dado erro no DBeaver, algum comando ficou só na tela do DBeaver e não foi copiado pro arquivo.

Commit:

```
git add material-alunos/database/schema.sql material-alunos/database/seed.sql
git commit -m "banco de dados: schema e seed da loja"
git push
```

5. Subir o backend (ainda com as rotas incompletas)

```
cd ../backend
python3 -m venv .venv
source .venv/bin/activate
# no Windows: .venv\Scripts\activate
pip install -r requirements.txt
python3 app.py
```

Acesse `http://127.0.0.1:5000/api/categorias` no navegador. Vai dar erro 501 dizendo que falta a query. Certo, é esperado. Cada rota vai responder 501 até você completar ela.

Se o `pip install` reclamar de "externally managed environment", é porque o Python do seu sistema bloqueia instalação fora de venv. A solução é a mesma dos comandos acima: sempre ativar o `.venv` antes de instalar ou rodar qualquer coisa nesse projeto.

6. Ligar as queries do bdcp2 nas rotas

Abra `material-alunos/backend/app.py`. Cada rota tem um comentário `# TODO` mostrando exatamente o que a query precisa devolver. A diferença entre a query que você já resolveu no bdcp2 e a que vai no `app.py` é sempre a mesma: no bdcp2 o valor era fixo (um id, um texto), aqui o valor vem de um parâmetro da rota e entra como `?`. Nunca concatene o valor direto na string SQL.

Tabela de conversão, tarefa do bdc2 até a rota do app.py:

bdc2	rota	o que muda
ecom04	GET /api/categorias	nenhuma mudança, cola direto
ecom05	GET /api/produtos (SELECT principal)	troca o WHERE/ORDER BY/LIMIT fixos pelas variáveis já prontas <code>where_sql</code> , <code>coluna</code> , <code>direcao_sql</code> e por <code>LIMIT ?</code> <code>OFFSET ?</code>
ecom06	GET /api/produtos (contagem)	troca o WHERE fixo por <code>where_sql</code>
ecom07	GET /api/produtos/destaques	troca <code>LIMIT 5</code> por <code>LIMIT ?</code>
ecom08	GET /api/produtos/<id>	troca <code>WHERE p.id = 1</code> por <code>WHERE p.id = ?</code>
ecom09	GET /api/produtos/<id>/avaliacoes	troca <code>WHERE a.produto_id = 1</code> por <code>?</code>
ecom10	POST /api/produtos/<id>/avaliacoes	usa o INSERT trocando os valores fixos por <code>?</code> ; falta escrever o SELECT que busca a avaliação recém-criada (<code>WHERE id = ?</code> , use <code>cursor.lastrowid</code>)
ecom11	POST /api/clientes	usa o INSERT com <code>?</code> ; falta a query que checa e-mail duplicado (<code>SELECT id FROM clientes WHERE email = ?</code>)
ecom12	POST /api/clientes/login	troca <code>email = 'x' AND senha = 'y'</code> por <code>email = ? AND senha = ?</code>
ecom13	GET /api/clientes/<id>/pedidos	troca <code>WHERE cliente_id = 1</code> por <code>?</code>
ecom14	usada dentro de <code>_validar_cupom</code> (duas rotas dependem dela)	troca <code>WHERE codigo = 'BEMVINDO10'</code> por <code>?</code>
ecom15	POST /api/pedidos	a mais trabalhosa. No bdc2 era um script fixo pra 2 itens. Aqui você separa em 4 comandos reutilizáveis: buscar produto por id, inserir o pedido, inserir um item, dar baixa no estoque. Os dois últimos rodam dentro de um loop, um por item do carrinho, porque agora o carrinho pode ter qualquer quantidade de produtos

bdc2	rota	o que muda
ecom16	GET /api/pedidos/<id>	troca WHERE ip.pedido_id = 4 por ? ; falta o SELECT do pedido em si, que segue o mesmo padrão das outras rotas
ecom17	GET /api/relatorios/resumo	cola direto; falta a segunda query, contar total de clientes (SELECT COUNT(*) AS total FROM clientes)
ecom18	GET /api/relatorios/mais-vendidos	troca LIMIT 5 por LIMIT ?
ecom19	GET /api/relatorios/faturamento	cola direto

São 15 rotas no total, mesmo número de tarefas de leitura/escrita que você fez no bdc2 mais os relatórios. Vá rota por rota, testando cada uma antes de passar pra próxima. Ordem sugerida e commits:

Bloco 1, catálogo (ecom04 a ecom09): implemente, teste cada uma no navegador ou no Insomnia comparando com docs/especificacao-api.md , depois:

```
git add material-alunos/backend/app.py
git commit -m "api: rotas de catalogo (categorias, produtos, destaques, avaliacoes)"
git push
```

Bloco 2, clientes (ecom10 a ecom13): cadastro, login, avaliação, pedidos do cliente. Teste cadastrando um cliente de mentira e logando com ele antes de comitar:

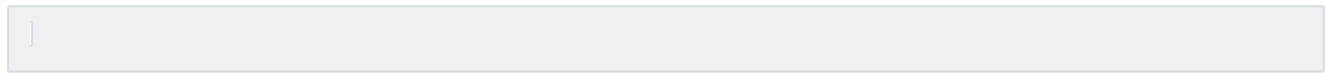
```
git add material-alunos/backend/app.py
git commit -m "api: cadastro, login e avaliacoes de cliente"
git push
```

Bloco 3, cupom e pedido (ecom14, ecom15, ecom16): essa é a rota mais importante do trabalho. Teste criando um pedido de verdade e confira no DBeaver (atualize a tabela com F5 e clique duas vezes pra ver os dados) que o estoque do produto baixou e que o pedido apareceu na tabela pedidos . Se o estoque não baixar ou o pedido gravar mesmo sem estoque suficiente, tem bug antes de comitar.

```
git add material-alunos/backend/app.py
git commit -m "api: validacao de cupom e criacao de pedido"
git push
```

Bloco 4, relatórios (ecom17 a ecom19): teste que os números batem com o que você vê direto no banco (SELECT SUM(total) FROM pedidos WHERE status != 'cancelado'; pra comparar com o faturamento do relatório, por exemplo).

```
git add material-alunos/backend/app.py
git commit -m "api: relatorios do painel da loja"
git push
```



Checklist de fim do Dia 1

- As 15 rotas respondem sem erro 501.
- Criar um pedido de verdade baixa o estoque do produto certo.
- Um pedido com estoque insuficiente é rejeitado (erro 400), sem gravar nada no banco.
- Um cupom vencido ou inativo é rejeitado na validação.
- Os 3 relatórios do painel mostram números coerentes com o que está no banco.